

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Shahanaj c	Reg. No.:	192372292
Section / Batch:	CSE AI	Date:	24/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
---------------	------------	-----------

Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1
----------------------------------	---	----

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I _____Shahanaj / 192372292_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____shahanaj_____

RUBRICS FOR CALCULATION

Criteria	Wt .	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25 %	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25 %	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization

Solution Development	25 %	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15 %	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10 %	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Program:

```
#include <iostream>
using namespace std;
```

```
int V;
int m;
int graph[20][20];
int color[20];
```

```
bool isSafe(int vertex, int c) {
for (int i = 0; i < V; i++) {
if (graph[vertex][i] == 1 && color[i] == c) {
return false;
}
}
return true;
}
```

```
bool graphColoringUtil(int vertex) {
```

```
if (vertex == V) {
return true;
}
```

```
for (int c = 1; c <= m; c++) {
if (isSafe(vertex, c)) {
color[vertex] = c;
if (graphColoringUtil(vertex + 1)) {
return true;
}
}
```

```

color[vertex] = 0;
}
}

return false;
}

void solveGraphColoring() {
for (int i = 0; i < V; i++) {
color[i] = 0;
}

if (graphColoringUtil(0)) {
cout << "Solution exists. Coloring assignment:\n";
for (int i = 0; i < V; i++) {
cout << "Vertex " << i << " -> Color " << color[i] << endl;
}
} else {
cout << "No solution exists with " << m << " colors\n";
}
}

int main() {
cout << "Enter number of vertices: ";
cin >> V;

cout << "Enter number of colors (m): ";
cin >> m;

cout << "Enter adjacency matrix (" << V << "x" << V << "):\n";
for (int i = 0; i < V; i++) {
for (int j = 0; j < V; j++) {
cin >> graph[i][j];
}
}

solveGraphColoring();

return 0;
}

```

Output:



SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

QUESTIONS

Q1 ✓
Graph Coloring Problem with M Colors
100 marks
1/1 answered**Q1 Graph Coloring Problem with M Colors** 100 marks

Develop a pseudocode using Backtracking to color the vertices of a graph using at most m colors such that adjacent vertices have different colors. Clearly identify inputs (graph, number of colors m) and output (valid coloring or failure). Design a logically correct algorithm that assigns colors vertex by vertex while checking adjacency constraints. Use loops, recursive calls, and conditional validation effectively during the coloring process. Apply pruning to eliminate invalid color assignments early. Ensure the pseudocode is clear, well-structured, and easy to follow. Handle all possible cases and guarantee correctness of the coloring result.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 int V;
5 int m;
6 int graph[20][20];
7 int color[20];
8 bool isSafe(int vertex, int c) {
9     for (int i = 0; i < V; i++) {
10         if (graph[vertex][i] == 1 && color[i] == c) {
11             return false;
12         }
13     }
14     return true;
15 }
```

Input

```
4
3
0 1 1 1
1 0 1 0
```

Output

```
Enter number of vertices: Enter
number of colors (m): Enter
adjacency matrix (4x4):
solution exists. Coloring
assignment:
Enter Back Color 1
```